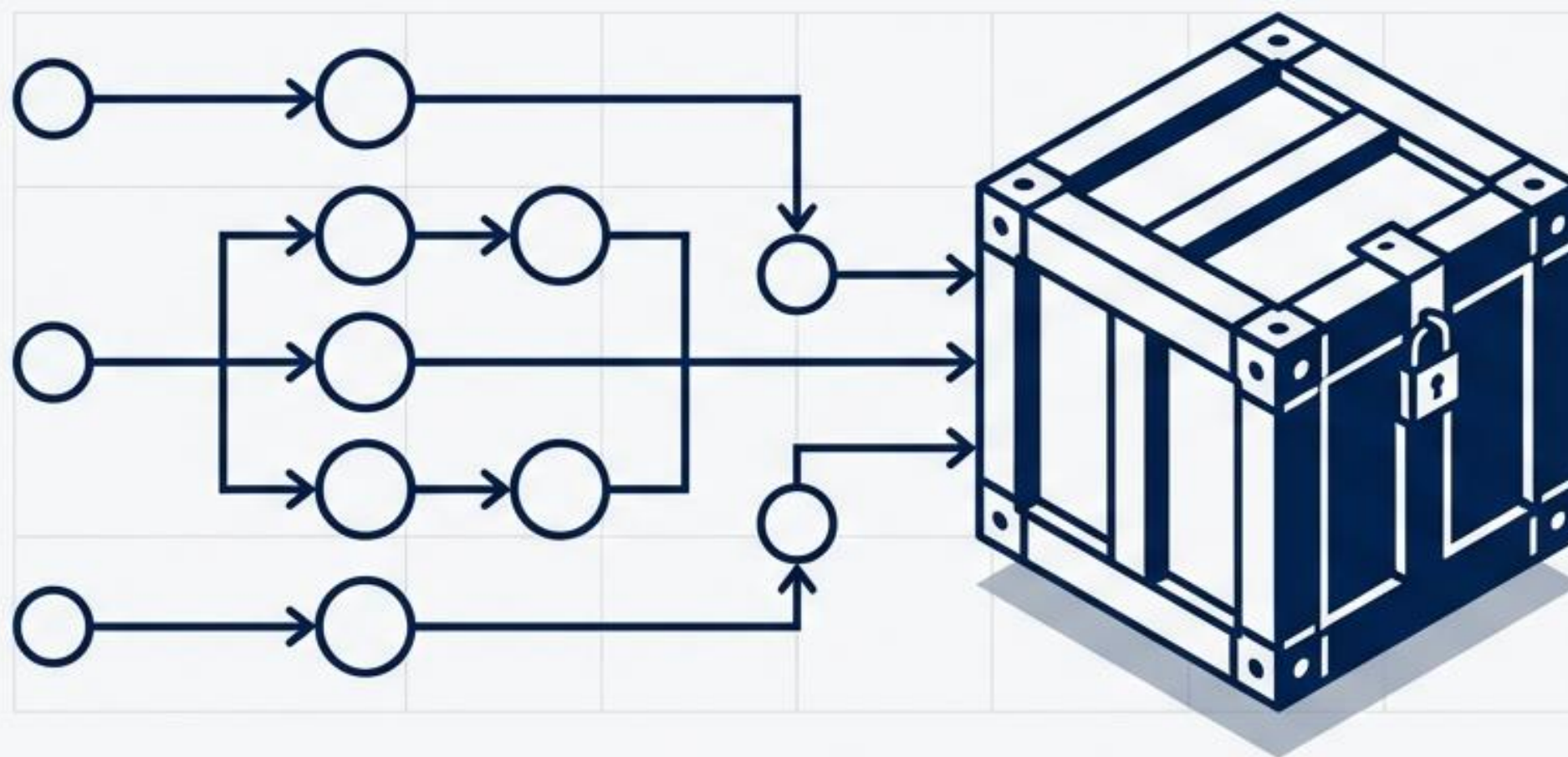


Debugging & Deployment

A. prof. Elsayed Mahdy

From Lab to Production

The AI Debugging and Deployment Playbook

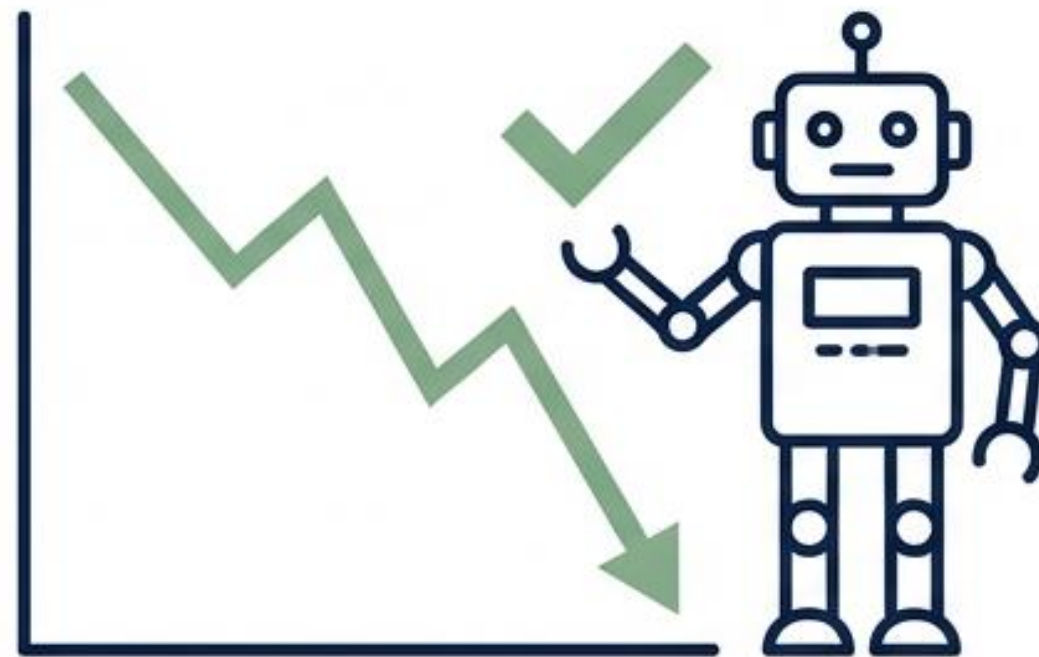


The AI Engineering Paradox



Traditional Code

Software fails loudly. Code crashes due to syntax or logic.



AI Models

AI fails silently. Bugs hide as bad accuracy, while the code runs perfectly.

Code
(syntax/logic)

Data
pipeline

Training
process

Evaluation

Deployment
behavior

The Fast Diagnosis Checklist

Confirm the Basics

- ☐ Is the code running without errors?
- ☐ Are shapes/dimensions correct?
- ☐ Are labels correct?
- ☐ Are you using the correct train/val/test split?

The Sanity Check (Must Do)

Action: Overfit a tiny dataset (e.g., 10 samples).

Rule: If the model can't reach near-perfect training accuracy, a bug exists.

- **Action:** Check loss decreases in the first few epochs/iterations.
- **Action:** Disable regularization/augmentation temporarily to simplify.

Diagnosing Data and Label Errors

Data Leakage (Most Common)

 **Symptom** Validation/test performance is unrealistically high, then fails in real use.

 **Cause** Test data used in preprocessing or feature selection.

 **Fix** Split data first, then fit scalers/encoders on training data only. Use pipelines.

Wrong Labels / Misaligned Data

 **Symptom** Model learns nonsense; accuracy stuck near random.

 **Fix** Print a few samples with their labels. Check dataset indexing.

Resolving Metric and Shape Mismatches

Wrong Metric

Symptom

“High accuracy” but the model is entirely useless (usually due to imbalanced data).

Fix

Use Precision, Recall, or F1-Score for imbalanced classification tasks.

Shape Mismatch (Silent Errors)

Symptom

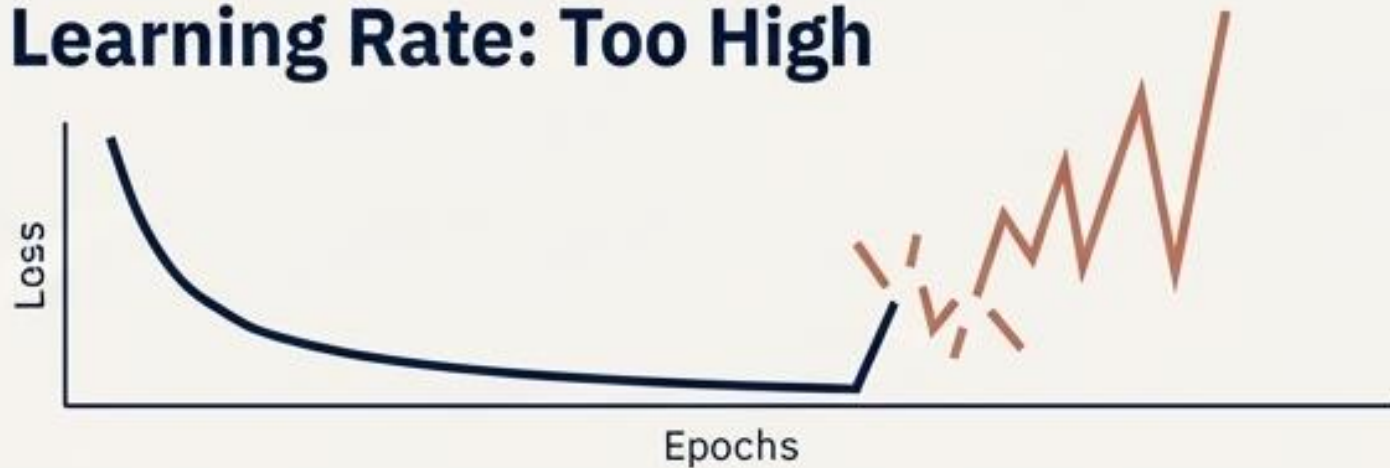
Training runs to completion, but the results are entirely wrong.

Fix

Print sizes of X, y, and predictions. Verify exactly one batch manually before training at scale.

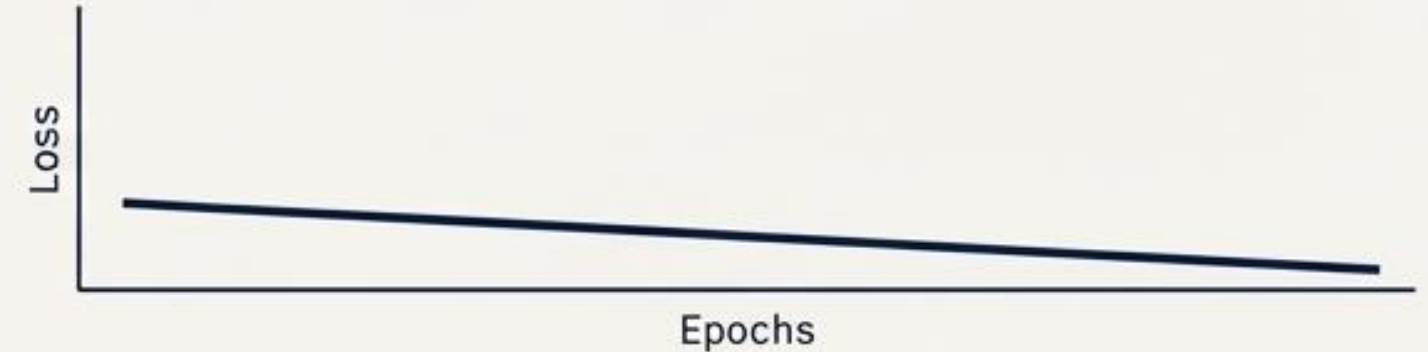
Tuning Learning Rates and Fitting

Learning Rate: Too High



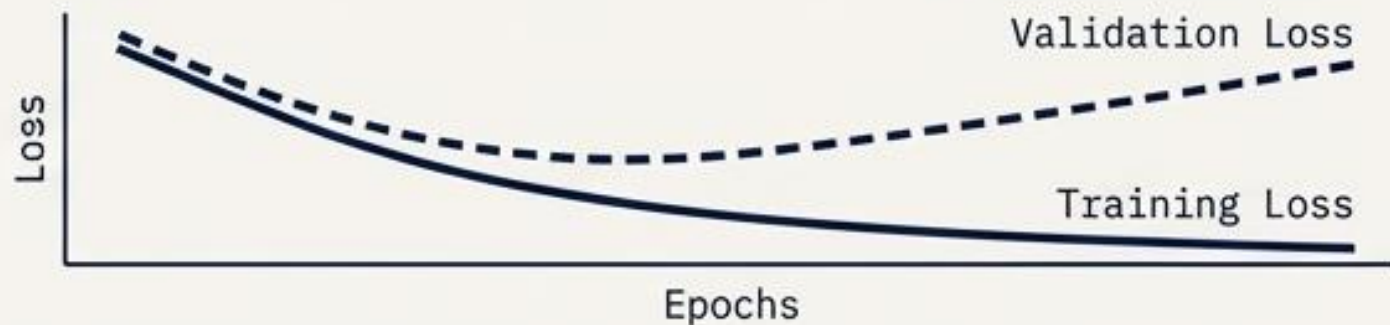
Loss **diverges or hits NaNs**. Try smaller learning rate.

Learning Rate: Too Low



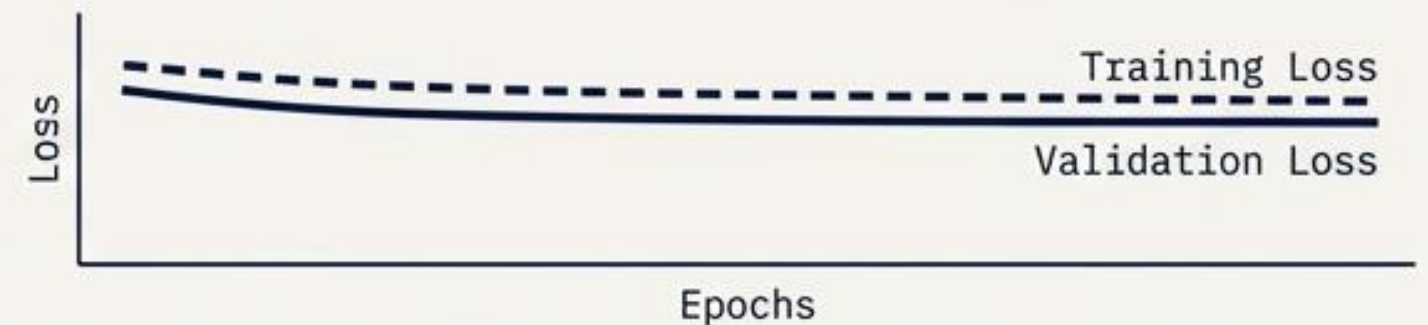
Training is extremely slow. Plot loss curve.

Generalization: Overfitting



Training good, validation bad.
Fix: **regularization, dropout, early stopping, more data**.

Generalization: Underfitting



Both training and validation bad.
Fix: **better features, more complex model, train longer**.

The Essential Debugging Toolkit



Print and Assert

- Simple but powerful. Print intermediate values.
- Use assertions to guarantee: no NaNs, correct ranges, correct shapes.



Always Log

- Loss per epoch
- Metrics per epoch
- Learning rate
- Learning rate
- Best model checkpoint

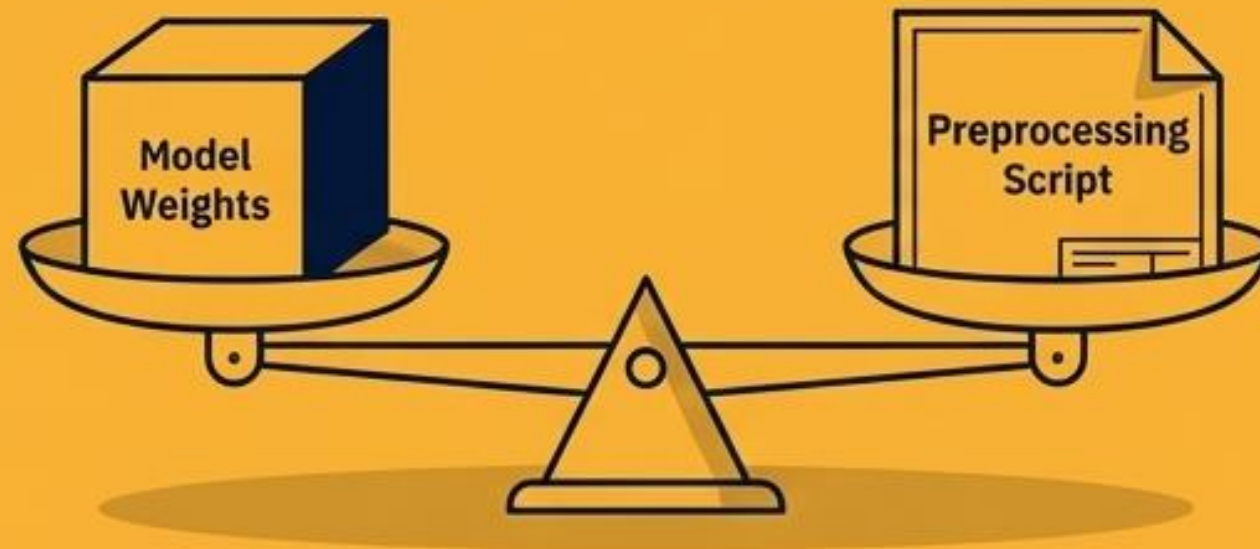


Visualize

- Training vs validation curves
- Confusion matrix
- Error distribution (for regression)

The Golden Rule of Deployment

Deployment means making your trained model usable by others. Never save just the weights.



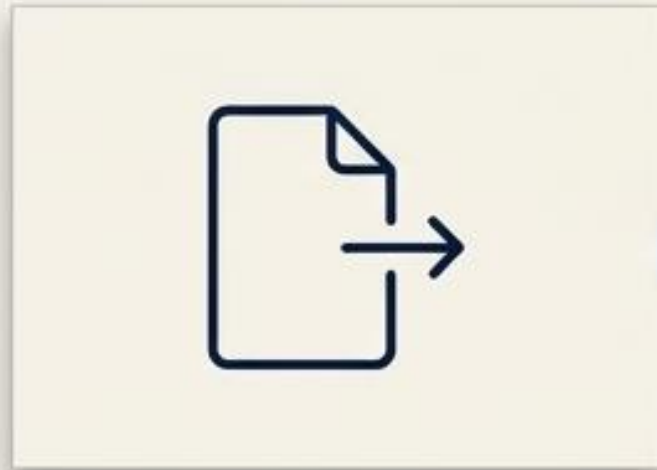
The Rule: Save model + preprocessing together.

The Bad: Saving only model weights.

The Good (The complete package):

- Trained model
- Scaler/Encoder
- Vocabulary/Tokenizer
- Class mapping
- Config (hyperparameters)

Choosing a Deployment Format



Step 1 - Script (Simplest)

Run `predict.py`. Takes a file or array input, outputs the predicted result.



Step 2 - Batch

Run model on many offline inputs simultaneously. Save results to CSV or JSON.



Step 3 - API (Common)

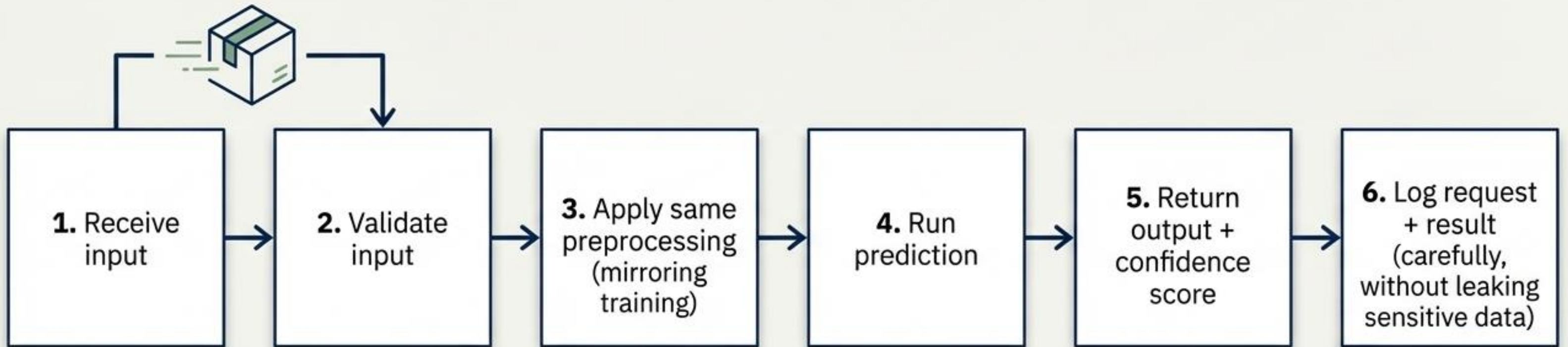
Expose a REST API. Send input -> return prediction. Built on frameworks like Flask or FastAPI.



Step 4 - Embedded/Edge

Deeply optimize the model for execution speed and strict memory limits.

The Production Inference Pipeline



Anticipating Production Risks



Model Drift

Data changes over time in the real world, causing model accuracy to silently drop.



Data Mismatch

The training data distribution is fundamentally different from the production environment.



Latency Issues

The model architecture is simply too slow for live user requirements.



Security and Privacy

Risk of exposing user data. Strict rule: Do not log sensitive raw inputs.

Continuous Post-Deployment Monitoring

99.9%

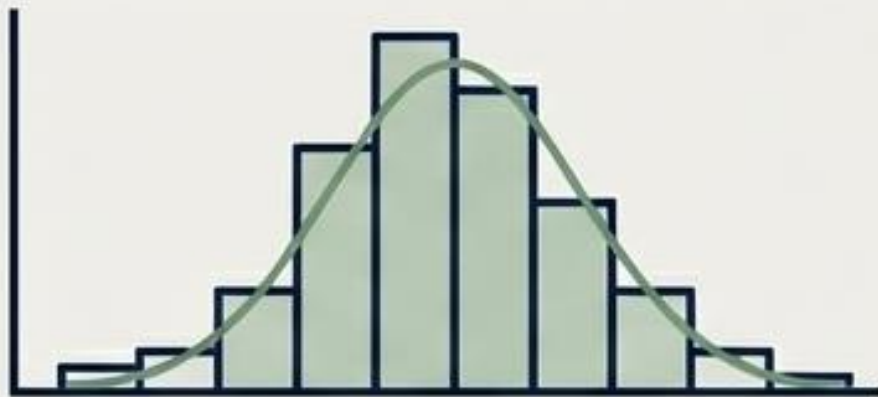
Number of requests (Volume)

45ms

Latency (Speed)

0.02%

Error rate (System health)

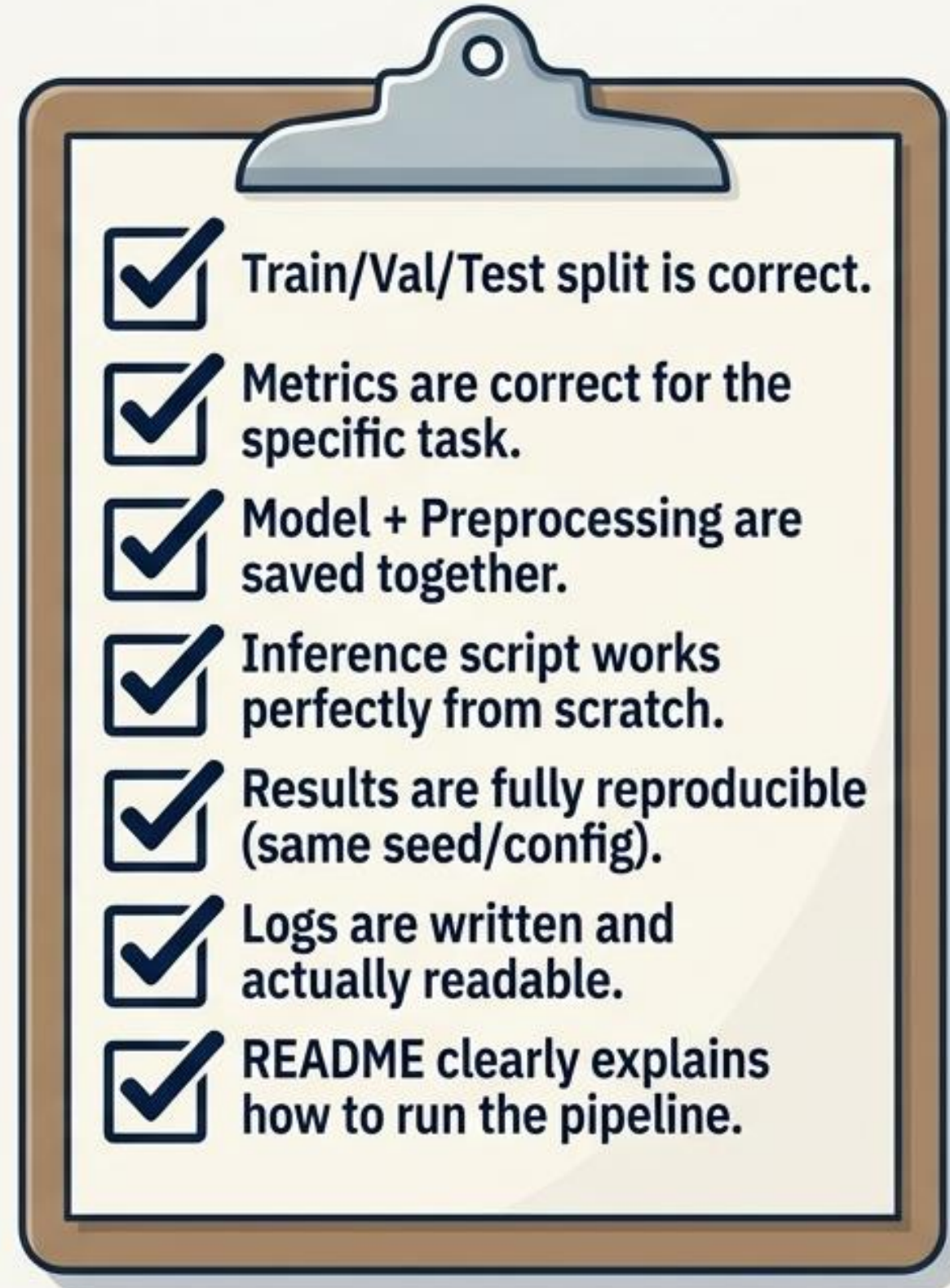


Confidence distribution
(Model certainty)



Performance on sampled labeled data
(Accuracy, if available)

The End-to-End Mastery Checklist



The Playbook Cheat Sheet

Debugging AI

- Perform sanity checks (overfit tiny data)
- Detect data leakage
- Verify labels & tensor shapes
- Tune learning rate
- Analyze errors

Deployment AI

- Save model + preprocessing package
- Define strict input/output interfaces
- Test inference completely from scratch
- Monitor for drift and latency continuously

THANK
you